PreCap: A Bounded, Self-Normalizing Activation Function that Matches SELU in Deep Normalization-Free MLPs

Aaron Schmitt

Independent researcher, Wiesbaden, Germany · ORCID 0009-0009-8689-5848

Preprint, version 0.1 · 26 October 2026 · Code and data: github.com/hermi-labs/precap-activation

Abstract. We introduce PreCap, a smooth, monotonic activation function that is bounded on both sides: an exponential unit that switches to its tangent line above a threshold (CapExp, which is exactly a shifted and scaled ELU) is applied after a soft tanh clip. SPreCap scales PreCap so that, like SELU, zero mean and unit variance are a fixed point, and adds an input gain b that sets the mean-field gradient gain χ_1. In a preregistered study on 50- and 100-layer MLPs without normalization or skip connections (MNIST and FashionMNIST, per-activation learning-rate tuning, 15 confirmation seeds, Holm correction), SPreCap-cs with χ_1 equal to SELU matches SELU: no cell is significantly worse, with median paired differences between -0.12 and +0.20 percentage points. Test error decreases monotonically as χ_1 approaches 1 in all four cells. The variant SPreCap-e03 ($\chi_1 = 1.03$) beats SELU on MNIST at depth 100 (5.28 % vs. 6.36 %, Holm-adjusted p = 0.0064, secondary test), while plain ELU is about as strong overall. PreCap is not more robust to the learning rate. Earlier large advantages at a fixed learning rate disappeared under fair tuning, and we report these negative results. We release fused CUDA and Apple MLX kernels that run within about 1.5× of native SELU in float32 and an installable package.

1 Introduction

Self-normalizing networks [1] showed that an activation function alone can keep the forward signal of deep fully connected networks at zero mean and unit variance. SELU achieves this with an unbounded positive branch. Bounded activations promise numerical safety, for example at low precision, and in deep networks without normalization layers, but classic bounded functions such as tanh saturate and are hard to train at depth [9]. Recent work replaces normalization layers with bounded point-wise functions (DyT [6], Derf [7], SALU [5]).

This work started from a simple idea: use the expressiveness of exponential growth as long as it is harmless, and cap it before anything explodes. It asks a narrow question. *Can a bounded, smooth exponential-type activation be made self-normalizing and match SELU in very deep MLPs without normalization?* Our contributions are:

- the PreCap/SPreCap family and its constants, including a third parameter that sets χ_1 (Section 2);
- a preregistered, hash-frozen evaluation with separate tuning and confirmation seeds, including all negative intermediate results (Sections 4–5);
- evidence that, for this bounded family, test error falls monotonically as $\chi_1 \to 1$, an independent confirmation of the slightly super-critical design in bSELU [2];
- fused forward/backward kernels for CUDA and Apple MLX and a pip-installable package (Section 6).

2 Definition

CapExp. For a threshold c, CapExp is exponential below c and continues along its tangent above it:

$$\mathrm{capexp}(w) = e^{w} - 1 \quad \text{for } w \le c, \qquad \mathrm{capexp}(w) = e^{c} - 1 + e^{c}\,(w - c) \quad \text{for } w > c$$

CapExp is exactly $e^{c}\cdot\mathrm{ELU}(w-c) + e^{c} - 1$ (verified numerically to $4\cdot10^{-15}$). With biases and a following linear layer it therefore represents the same functions as ELU [8]. The new element is the soft clip in front of it.

PreCap and SPreCap. With the clip $w = s\cdot\tanh(bx/s)$, fixed $s = 1.5$ and $c = 1$:

$$f(x) = \lambda \cdot \{1 \text{ if } x > 0; \quad \alpha \text{ if } x \le 0\} \cdot \mathrm{capexp}(s\tanh(bx/s))$$

PreCap0 is $\lambda = \alpha = b = 1$. It has slope 1 at the origin and is close to $2\cdot\mathrm{SiLU}$ for small inputs. For SPreCap, (λ, α) are chosen as in SELU so that $\mathrm{E}[f(z)] = 0$ and $\mathrm{E}[f(z)^2] = 1$ for $z \sim \mathrm{N}(0, 1)$, computed with Gauss–Hermite quadrature. The gain b sets $\chi_1 = \mathrm{E}[f'(z)^2]$, which controls how gradients grow from the output to the input layer at initialization: $\log_{10}(g_1/g_L) \approx (L-1)/2 \cdot \log_{10} \chi_1$ [3, 9]. Since tanh is bounded, the output is bounded on both sides (Table 1). The function is monotonic and continuous, its derivative jumps at 0 by the factor α as in SELU, and its second derivative is discontinuous at $w = c$.

Variant	λ	α	b	χ_1	Range
SPreCap	0.80676869644	2.39695130922	1	1.1305	[−1.50, 2.48]
SPreCap-cs	1.102702500	2.053776267	0.71188112	1.0716 (= SELU)	[−1.76, 3.39]
SPreCap-e03	1.985552225	1.608136397	0.4118896262	1.0300	[−2.48, 6.11]
PreCap0	1	1	1	–	[−0.78, 3.08]

Table 1. Constants (s = 1.5, c = 1). Derivations: `math/p1_spreccap_exact.py`, `math/p3a3_chi1_family.py`.

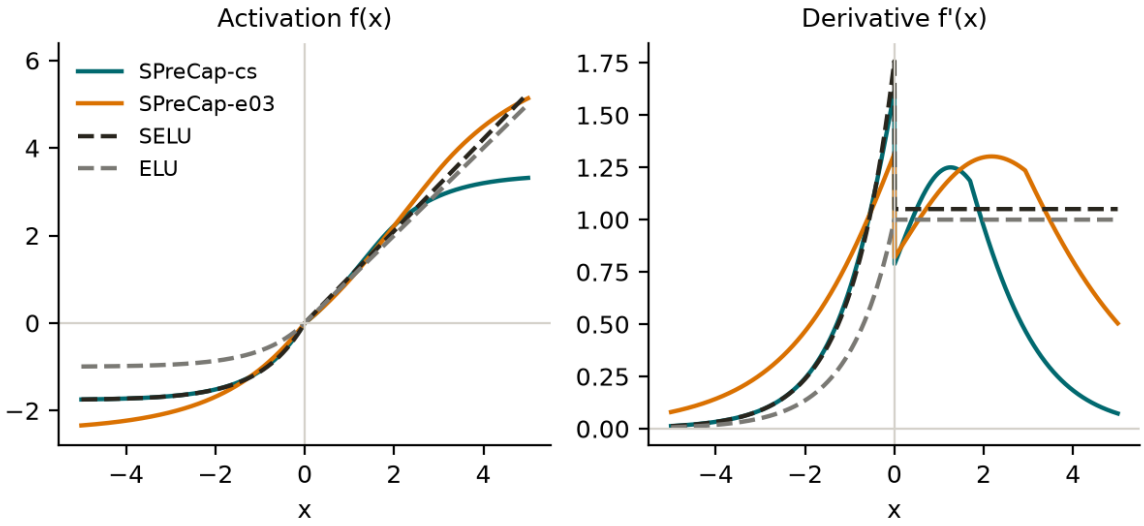


Figure 1. SPreCap-cs and SPreCap-e03 compared with SELU and ELU. Unlike SELU and ELU, the PreCap variants saturate for large positive inputs.

3 Related work

Self-normalization. SELU [1] introduced the mean/variance fixed point. Chen et al. [2] generalize it with explicit tolerances, $\mathrm{E}[f] = \varepsilon_\mu$, $\mathrm{E}[f^2] = 1 - \varepsilon_q$, $\varphi(1) = 1 + \varepsilon_\varphi$, and report that bSELU with a slightly super-critical gradient term beats SELU on a 56-layer CNN without shortcuts (88.54 % vs. 84.81 % on CIFAR-10). Our χ_1 family follows the same idea for a bounded function. Lu et al. [4] obtain bidirectional self-normalization with Gauss–Poincaré normalized activations and orthogonal weights. **Bounded and normalization-free.** DyT [6] and Derf [7] replace normalization layers in Transformers with $\tanh(\alpha x)$ and $\mathrm{erf}(\alpha x + s)$. SaluNet [5] uses the bounded, learnable $\mathrm{SALU}(x) = ax/\sqrt{1 + abx^2}$ to train ResNets and Transformers without normalization. **Signal propagation.** Edge-of-chaos theory [3] links $\chi_1 = 1$ to trainable depth. Bukva et al. [9] show that saturation can impede training even at criticality, which matches our observation that the critical point of PreCap0 is fragile. **Similar shapes.** TanhExp, $x \cdot \tanh(e^x)$ [10], combines tanh and exp in the opposite order and is unbounded. A survey of over 400 activations [11] contains no identical function. To our knowledge the exact PreCap form is new, but every building block is known. The contribution is therefore small and specific.

4 Experimental protocol

Every study was preregistered before it ran: hypotheses, tests, decision rules, and SHA-256 hashes of all scripts. Hashes were verified before and after each run. Tuning seeds and confirmation seeds are disjoint across all studies. Comparisons are paired by seed and use an exact sign-flip permutation version of the Wilcoxon signed-rank test, which does not depend on the scipy version. Holm correction is applied per test family.

Phase 3a-3 (main study). MLP, width 256, depth 50 and 100, no normalization, no residual connections; MNIST and FashionMNIST, standardized inputs, the last 5000 training images as a fixed validation split; Adam, batch 128, 1720 steps; weights $W \sim \mathrm{N}(0, g^2/\mathrm{fan_in})$ with a variance-preserving gain $g = 1/\sqrt{\mathrm{E}[f(z)^2]}$ for every activation (≈ 1 for SELU and SPreCap). Learning rates $\{1, 2, 5\} \cdot 10^{-5}$, $\{1, 2, 5\} \cdot 10^{-4}$; the rate with the highest median validation accuracy over 4 tuning seeds is selected per activation and cell, then evaluated on 15 confirmation seeds. Activations: SPreCap, SPreCap-cs, SPreCap-e03, SPreCap-cs with a learnable clip s per layer, SELU, and ELU. In total 936 runs. All runs were finite, and the analysis was reproduced independently from the raw data.

Primary hypothesis and v0.1 decision rule. SPreCap-cs vs. SELU in four cells, Holm over 4. SPreCap-cs counts as validated if it is significantly worse in no cell and the median difference is at most +0.5 percentage points in at least 3 of 4 cells. Secondary tests (24, one Holm family) include e03 vs. SELU and cs vs. ELU.

5 Results

Cell (data, depth)	SPreCap	SPreCap-cs	cs, learn. *s*	SPreCap-e03	SELU	ELU
FashionMNIST, 50	16.15	14.92	14.83	**14.57**	14.69	14.67
FashionMNIST, 100	24.21	16.98	17.02	**16.61**	17.29	16.62
MNIST, 50	5.13	4.42	4.01	4.05	4.31	**3.75**
MNIST, 100	13.56	6.27	5.92	**5.28**	6.36	5.78

Table 2. Median test error (%) over 15 confirmation seeds at the selected learning rate. Best per row in bold.

Cell	cs − SELU (pp)	cs better / worse	Holm p	e03 − SELU (pp)	Holm p
FashionMNIST, 50	+0.20	4 / 11	0.333	+0.16	1.000
FashionMNIST, 100	-0.12	9 / 6	1.000	-0.74	0.123
MNIST, 50	+0.12	7 / 8	1.000	-0.37	0.360
MNIST, 100	+0.02	7 / 8	1.000	-1.33	**0.0064**

Table 3. Paired tests. Differences are medians of per-seed differences, negative means lower error than SELU. Left: primary family (Holm over 4). Right: secondary family (Holm over 24).

Primary result. SPreCap-cs is significantly worse than SELU in none of the four cells, and all median differences are within ±0.2 pp, so the preregistered v0.1 rule is met. It is not better than SELU either: all four primary tests are inconclusive. In the secondary family SPreCap-cs is better than the original SPreCap in 4 of 4 cells and worse than ELU on MNIST at depth 50 (+0.58 pp, Holm p = 0.022). SPreCap-e03 is better than SELU on MNIST at depth 100 (-1.33 pp, 13 of 15 seeds) and better than SPreCap-cs there. Mean ranks over the four cells: e03 1.5, ELU 1.75, learnable *s* 3.25, cs 4.25, SELU 4.25, SPreCap 6.0.

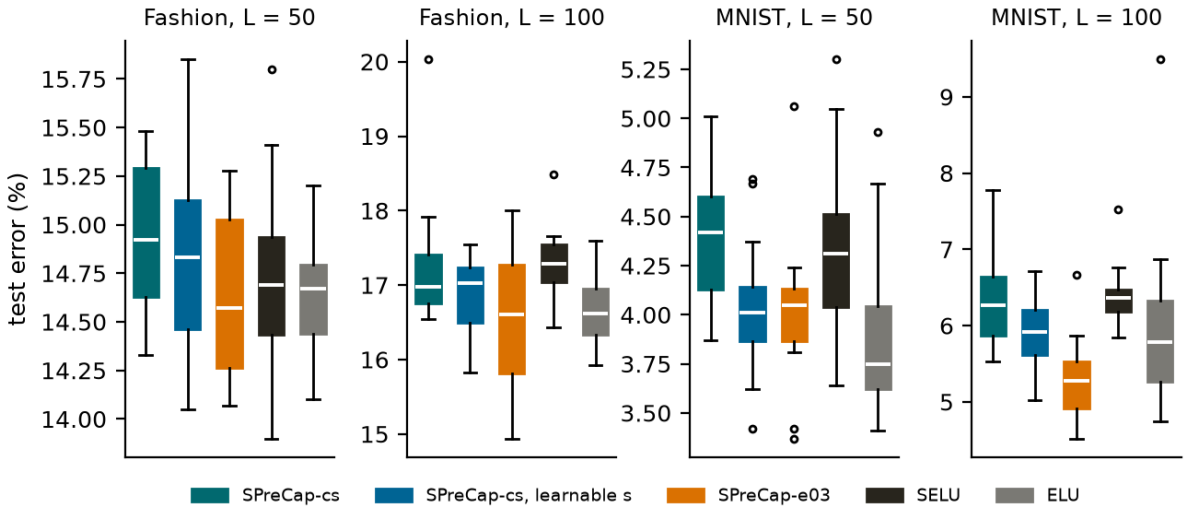


Figure 2. Test error per confirmation seed (15 seeds) at the selected learning rate. The original SPreCap ($\chi_1 = 1.13$) lies far above and is omitted, see Figure 3.

χ_1 dose-response. Within the family, the median error decreases strictly as χ_1 goes from 1.13 to 1.07 to 1.03 in all four cells (Figure 3). The largest gain comes from 1.13 to 1.07. Measured gradient growth at initialization agrees with the mean-field prediction to within 0.17 orders of magnitude (Figure 4), and SPreCap-cs is indistinguishable from SELU at initialization by construction.

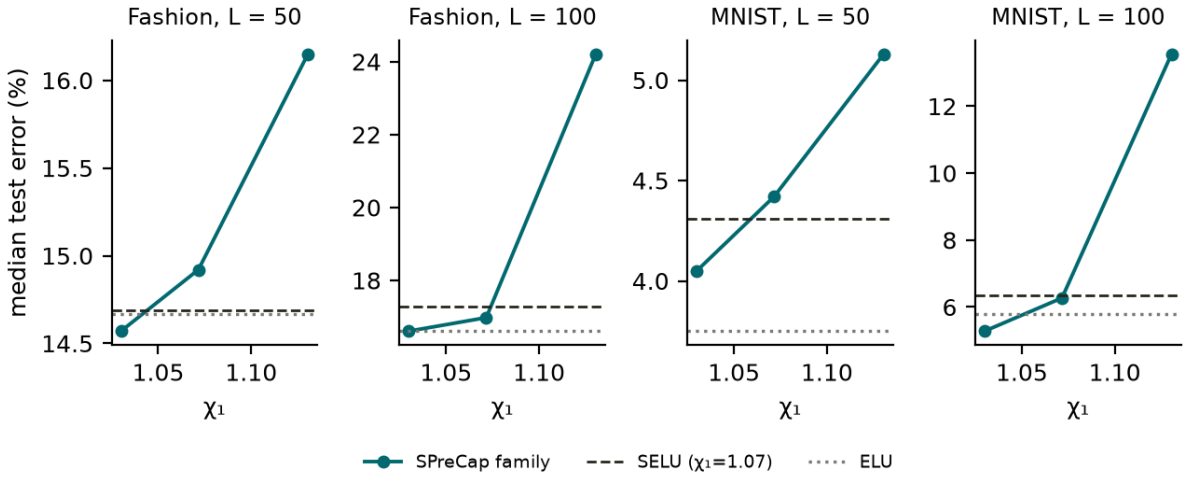


Figure 3. Median test error of the SPreCap family against χ_1, with SELU and ELU for reference.

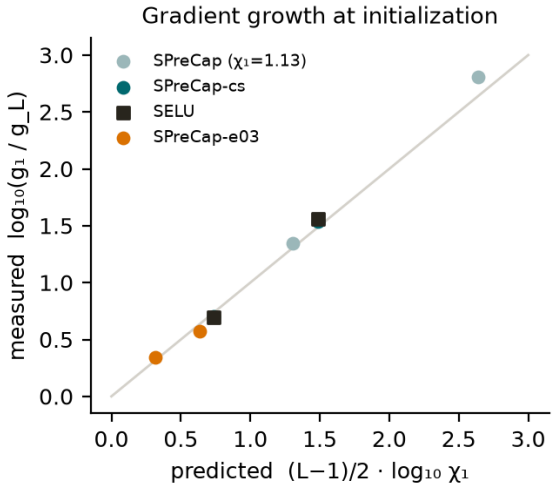


Figure 4. Gradient growth from the last to the first layer at initialization, at depth 50 (lower point) and 100 (upper point), against the prediction $(L-1)/2 \cdot \log_{10} \chi_1$. The line marks equality.

Learning rate. PreCap is not more tolerant of large learning rates (Figure 5). At depth 50 and learning rate $5 \cdot 10^{-4}$, SELU and ELU still reach 59–72 % validation accuracy, all PreCap variants 10–15 %. At depth 100 only the original SPreCap still learns at 10^{-4}. This shows a trade-off: larger χ_1 tolerates larger steps but reaches a worse optimum. The learnable clip s moved to a median of about 1.26–1.34 and helped only on MNIST at depth 50.

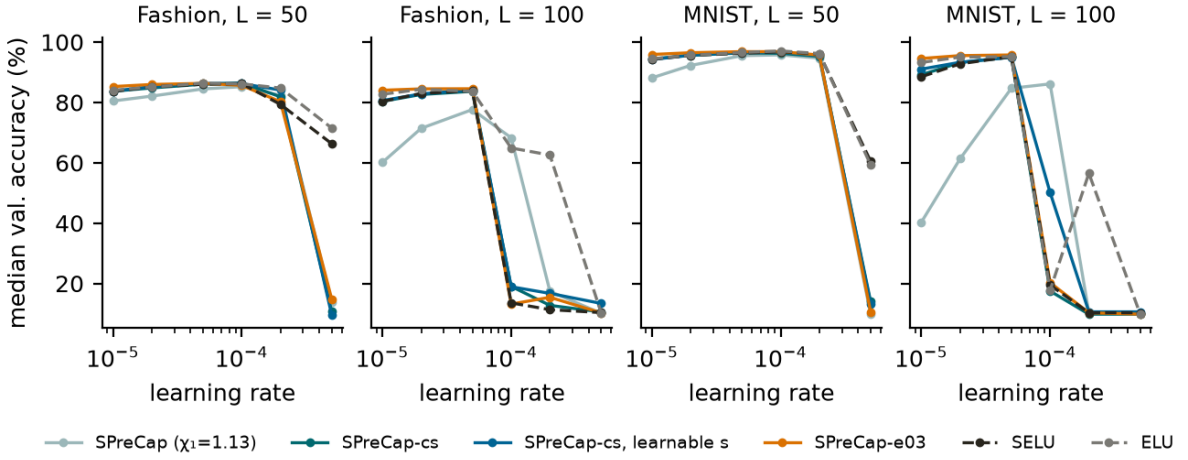


Figure 5. Median validation accuracy over the 4 tuning seeds for every learning rate in the grid.

History and negative results. The project began with 1D, 2D and 3D regression tasks (more than 19,000 runs). At a fixed learning rate of 10^{-3}, PreCap was far better than SiLU and GELU in residual networks. This advantage vanished once every activation received its own tuned learning rate (0 of 30 tests better, 2 worse), and a 3D transfer study (7200 runs) confirmed this. The explanation is scale: PreCap has slope 1 at the origin, SiLU and GELU 0.5. One robust property remained. The bound keeps undamped 128-layer residual networks trainable where SiLU and GELU diverge. In the first deep-MLP study (Phase 3a, 2430 runs) PreCap0 failed to learn at depth 100. In Phase 3a-2 (1092 runs) the original SPreCap was significantly worse than SELU in 4 of 4 cells. Its larger χ_1 (1.13 vs. 1.07) inflated input-side gradients by about 600× at depth 100, compared with 35× for SELU. This diagnosis motivated the χ_1 family above. All of these studies, including the failed ones, are documented in the repository.

6 Kernels and package

An unfused PreCap needs about ten element-wise operations per direction. We provide fused forward and backward kernels: CUDA C++ compiled at run time through `torch.cuda.jiterator` (no CUDA toolkit required) and a Metal kernel through `mx.fast.metal_kernel` with a custom VJP for MLX. On an RTX 4070 all 17 checks pass (values and gradients in fp32/fp64, gradcheck, edge values up to $\pm 10^4$, a 20-layer MLP trained with kernel vs. reference), with a maximum relative fp32 error of $1.1 \cdot 10^{-6}$. A second, extended run on the same GPU passes 14 further checks: fp16 and bf16 values and gradients for all four presets (maximum relative error $3.8 \cdot 10^{-4}$ in fp16 and $2.9 \cdot 10^{-3}$ in bf16, i.e. at the rounding level of the format), NaN propagation, ±Inf mapped to the two bounds, non-contiguous tensors, and identical results of the package module and the reference. Speed varies between runs: forward plus backward over 16M fp32 elements took 0.91 ms vs. 0.89 ms for native SELU in the first run (Figure 6) and 1.06 ms vs. 0.73 ms in the second, where ReLU, SiLU, GELU and ELU also took 0.73 ms, so the fused kernel costs up to about 1.5× a native activation. In fp16 the kernel is slower (1.61 ms vs. 0.34 ms for SELU) because inputs are converted to fp32 and back in separate passes. On an M1 Pro all 12 checks pass. The package installs with

```
pip install "precap[torch] @ git+https://github.com/hermi-labs/precap-activation"
```

and exposes `from precap import PreCap` (the CUDA kernel is used automatically on CUDA tensors) and `from precap.mlx import PreCap`.

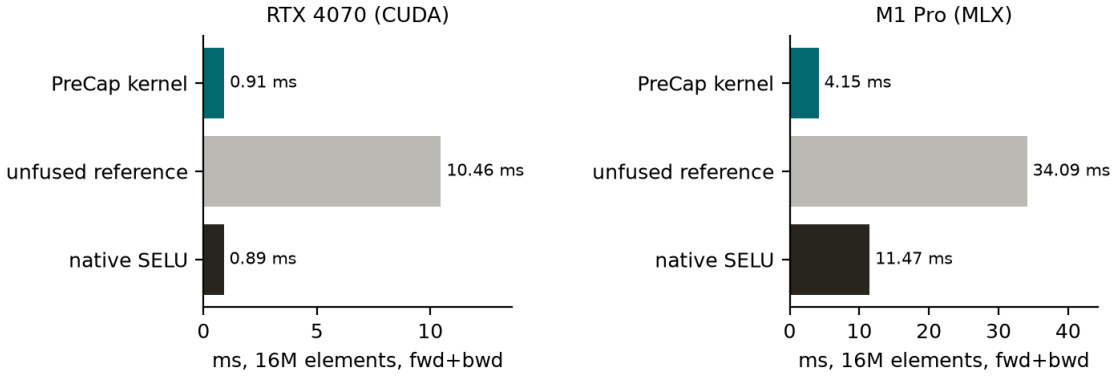


Figure 6. Time for forward plus backward over 16M float32 elements. The MLX measurement ran alongside a training job and is likely pessimistic.

7 Limitations

Only fully connected networks on MNIST and FashionMNIST, one width (256), one optimizer (Adam) and short training (1720 steps). No CNNs, Transformers, or large datasets. The advantage of SPreCap-e03 rests on one cell of a secondary family and should be treated as a hypothesis. The study was not powered to separate e03 from ELU. The benefit of a bounded range for fp16/fp8/int8 inference is plausible but was not measured. The CUDA kernel has no native fp16/bf16 path and is about 4.7× slower than native SELU in fp16. The work was done by a single independent researcher on consumer hardware with AI assistance for code and analysis (see acknowledgments).

8 Conclusion

A bounded, smooth exponential-type activation can be made self-normalizing and then matches SELU in 50- and 100-layer MLPs without normalization. The decisive quantity is not the bound but the gradient gain χ_1: a value slightly above 1 gives the best results, in line with bSELU. PreCap is not a general improvement over SELU or ELU. It is a bounded alternative with the same accuracy in this setting, open for tests where a bounded range should matter: low-precision inference, normalization-free deep networks, and depth-dependent $\chi_1 = 1 + k/L$.

Acknowledgments. Idea, function and all experiments by the author. Implementation, statistical analysis, literature research and writing were supported by Perplexity Computer with Claude models by Anthropic. See `docs/Danksagung_und_Werkzeuge.md`.

Reproducibility. Preregistrations, frozen scripts with SHA-256 hashes, raw data (or hashes), analysis outputs, figure and paper sources (`paper/make_figures.py`, `paper/build_paper.py`) are in the repository under Apache-2.0.

References

[1] Klambauer, Unterthiner, Mayr, Hochreiter. Self-Normalizing Neural Networks. NeurIPS 2017. https://arxiv.org/abs/1706.02515

[2] Chen, Zhao, Deng, Ding, Wen, Li, Xie. Large-scale self-normalizing neural networks (bSELU). Journal of Automation and Intelligence 3, 101–110, 2024. https://www.sciopen.com/article/10.1016/j.jai.2024.05.001

[3] Hayou, Doucet, Rousseau. On the Impact of the Activation Function on Deep Neural Networks Training. ICML 2019. https://arxiv.org/abs/1902.06853

[4] Lu, Gould, Ajanthan. Bidirectionally Self-Normalizing Neural Networks. 2020/2021. https://arxiv.org/abs/2006.12169

[5] Zaied. SaluNet: Enabling Total Plasticity in Normalization-Free Networks. 2026. https://arxiv.org/abs/2606.02927

[6] Zhu, Chen, He, LeCun, Liu. Transformers without Normalization (DyT). CVPR 2025. https://arxiv.org/abs/2503.10622

[7] Chen, Lu, Zhu, Sun, Liu. Stronger Normalization-Free Transformers (Derf). 2025. https://arxiv.org/abs/2512.10938

[8] Clevert, Unterthiner, Hochreiter. Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). ICLR 2016. https://arxiv.org/abs/1511.07289

[9] Bukva, de Gier, Grosvenor, Jefferson, Schalm, Schwander. Criticality versus uniformity in deep neural networks. 2023. https://arxiv.org/abs/2304.04784

[10] Liu, Di. TanhExp: A Smooth Activation Function with High Convergence Speed for Lightweight Neural Networks. IET Computer Vision. https://arxiv.org/abs/2003.09855

[11] Kunc, Kléma. Three Decades of Activations: A Comprehensive Survey of 400 Activation Functions for Neural Networks. 2024. https://arxiv.org/abs/2402.09092